Java Backend Architecture

Interview Series

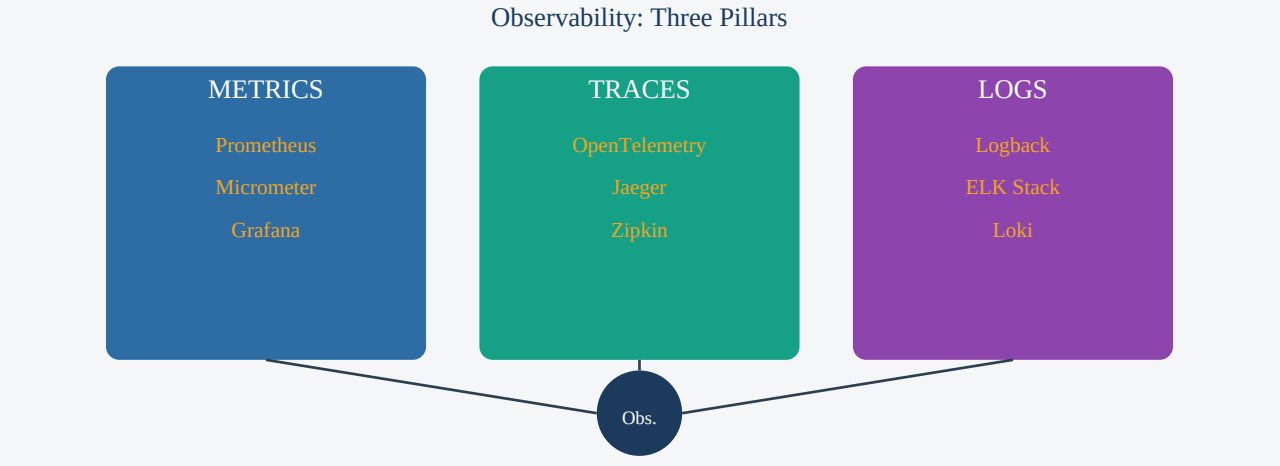
Chapter 14.8 Observability - Metrics, Tracing & Logging

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 14.8 - Observability: Metrics, Tracing & Logging

Overview

Observability is the ability to understand a system's internal state from its external outputs. The three pillars are: Metrics (aggregated numerical measurements), Traces (end-to-end request flows), and Logs (timestamped event records). Together they provide full visibility into distributed systems.



Metrics - Prometheus & Micrometer

Prometheus	Pull-based metrics system: scrapes /actuator/prometheus endpoint every N seconds. Time-series DB with PromQL query language. Alertmanager for threshold-based alerts.
Micrometer	Instrumentation facade for JVM metrics (like SLF4J for metrics). Vendor-neutral API; backends: Prometheus, Datadog, New Relic. Spring Boot auto-configures via spring-boot-actuator.
Grafana	Visualization layer: connects to Prometheus data source, renders dashboards with panels (graphs, gauges, tables). Alerting via Grafana Alert Manager.
Counter	Monotonically increasing count (e.g., http_requests_total). Never decreases.
Gauge	Current value that can go up/down (e.g., jvm_memory_used_bytes, active_connections).
Histogram	Distribution of observations in configurable buckets (e.g., request latency: le=0.1, le=0.5, le=1.0). Enables percentile calculation.
Summary	Pre-computed quantiles (p50, p95, p99) on client side. Less flexible than histogram but cheaper server-side.

Spring Boot Micrometer + Prometheus Setup

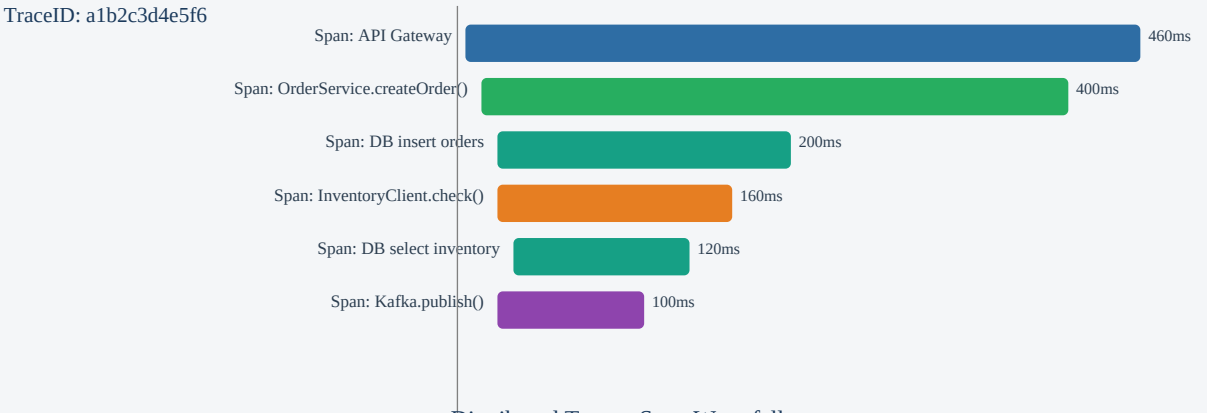
```
// build.gradle implementation "org.springframework.boot:spring-boot-starter-actuator"
implementation "io.micrometer:micrometer-registry-prometheus" // application.yml management:
endpoints: web: exposure: include: health,info,prometheus,metrics metrics: tags: application:
order-service env: production // Custom metrics in service @Service public class OrderService {
private final Counter orderCounter; private final Timer orderTimer; public
```

```
OrderService(MeterRegistry registry) { this.orderCounter = Counter.builder("orders.created")
.tag("type", "standard") .register(registry); this.orderTimer =
Timer.builder("orders.processing.time") .publishPercentiles(0.5, 0.95, 0.99)
.register(registry); } public Order createOrder(OrderRequest req) { return
orderTimer.record(() -> { Order o = processOrder(req); orderCounter.increment(); return o; });
} }
```

Useful PromQL Queries

```
# Request rate (per second, 5-min window)
rate(http_server_requests_seconds_count{application="order-service"}[5m]) # Error rate
percentage sum(rate(http_server_requests_seconds_count{status=~"5.."}[5m])) /
sum(rate(http_server_requests_seconds_count[5m])) * 100 # 99th percentile latency
histogram_quantile(0.99, rate(http_server_requests_seconds_bucket{uri="/api/orders"}[5m]) ) #
JVM heap usage jvm_memory_used_bytes{area="heap"} / jvm_memory_max_bytes{area="heap"} * 100
```

Distributed Tracing - OpenTelemetry & Jaeger



Distributed Trace - Span Waterfall

Tracing Concepts

Trace	Represents a single end-to-end request flow through the entire system. Identified by a unique traceId.
Span	A single unit of work within a trace (e.g., one service call, DB query). Has spanId, parent spanId, start/end time, tags, logs.
TraceId	128-bit unique identifier for a trace, propagated through all services via HTTP headers.
SpanId	64-bit unique identifier for a specific span within a trace.
W3C traceparent	Standard HTTP header for trace context propagation: traceparent: 00-{traceId}-{spanId}-{flags}
OpenTelemetry	CNCF standard SDK + protocol (OTLP) for traces, metrics, and logs. Vendor-neutral; exports to Jaeger, Zipkin, Datadog, etc.
Jaeger	Distributed tracing backend by Uber/CNCF: stores spans, provides UI to visualise trace waterfalls, find slow spans.

OpenTelemetry Java Agent Setup

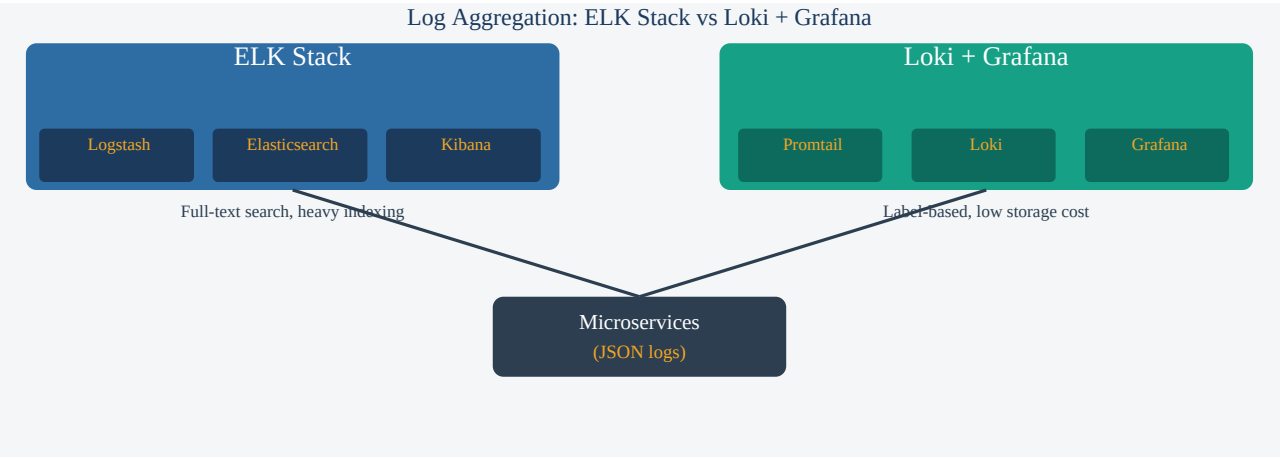
```
# JVM flag - zero-code auto-instrumentation java -javaagent:opentelemetry-javaagent.jar \
-Dotel.service.name=order-service \ -Dotel.exporter.otlp.endpoint=http://jaeger:4317 \
-Dotel.traces.exporter=otlp \ -Dotel.metrics.exporter=prometheus \ -jar order-service.jar // Manual span creation @Service public class OrderService { private final Tracer tracer = GlobalOpenTelemetry.getTracer("order-service"); public Order processOrder(OrderRequest req) { Span span = tracer.spanBuilder("processOrder") .setAttribute("order.userId", req.getUserId()) .startSpan(); try (Scope scope = span.makeCurrent()) { Order order = doProcess(req); span.setAttribute("order.id", order.getId().toString()); return order; } catch (Exception e) { span.setStatus(StatusCode.ERROR, e.getMessage()); throw e; } finally { span.end(); } } }
```

Trace Context Propagation

- OpenTelemetry auto-instruments RestTemplate, WebClient, Feign, Kafka, JDBC
- Propagates W3C traceparent header across HTTP calls automatically
- For async threads: use Context.current().wrap(runnable) to propagate context

• For Kafka: OTel auto-instruments producers/consumers to propagate via message headers

Structured Logging & Log Aggregation



Structured Logging with Logback + MDC

```
// logback-spring.xml - JSON structured logging <configuration> <appender name="JSON"
class="ch.qos.logback.core.ConsoleAppender"> <encoder
class="net.logstash.logback.encoder.LogstashEncoder">
<includeMdcKeyName>traceId</includeMdcKeyName> <includeMdcKeyName>spanId</includeMdcKeyName>
<includeMdcKeyName>userId</includeMdcKeyName> </encoder> </appender> <root
level="INFO"><appender-ref ref="JSON"/></root> </configuration> // Service - MDC correlation
@RestController public class OrderController { @PostMapping("/orders") public
ResponseEntity<Order> create(@RequestBody OrderRequest req, @RequestHeader("X-User-Id") String
userId) { MDC.put("userId", userId); // traceId auto-injected by OTel bridge try {
log.info("Creating order for user {}", userId); Order order = orderService.createOrder(req);
log.info("Order created orderId={}", order.getId()); return ResponseEntity.ok(order); }
finally { MDC.clear(); } } }
```

ELK Stack vs Loki + Grafana

Aspect	ELK Stack	Loki + Grafana
Indexing	Full-text index on all fields (heavy)	Label-based index only (lightweight)
Query language	Elasticsearch DSL / KQL	LogQL (label filters + regex)
Storage cost	High (indexes replicated)	Low (compressed log chunks)
Search power	Full-text search, aggregations	Filter by labels, then grep lines
Integration	Logstash for parsing, Kibana UI	Promtail/Fluentd, Grafana UI
Best for	Complex log analytics, compliance	Simple log viewing, cost-conscious

Log Levels and Best Practices

ERROR	System failures requiring immediate attention. Always include exception stack trace.
WARN	Unexpected situations that did not cause failure but need investigation (retry succeeded, deprecated API used).
INFO	Normal operational events: service start/stop, significant business events (order created), config loaded.

DEBUG	Detailed diagnostic info for development/troubleshooting. Disable in production or use dynamic log level.
MDC	Mapped Diagnostic Context: ThreadLocal key-value store injected into every log line. Use for: traceId, userId, requestId, tenantId.

Interview Q&A - Observability: Metrics, Tracing & Logging

Q1. What are the three pillars of observability?

Metrics: aggregated numerical measurements over time (request rate, error rate, latency percentiles, CPU usage). Traces: end-to-end request flow through distributed services, showing which service took how long. Logs: timestamped text records of events. Together they answer: what is happening (metrics), where it is slow (traces), and why it failed (logs).

Q2. What is Prometheus and how does it collect metrics?

Prometheus is a pull-based time-series metrics system. It scrapes HTTP endpoints (e.g., /actuator/prometheus) at regular intervals (default 15s). Each scrape collects all registered metrics as key=value pairs with labels. Data is stored in a local time-series DB with a configurable retention period. Alertmanager integrates to send alerts when metric thresholds are breached.

Q3. What is Micrometer and why use it over direct Prometheus client?

Micrometer is a vendor-neutral instrumentation facade, like SLF4J but for metrics. You write instrumentation code once against the Micrometer API; the backend (Prometheus, Datadog, New Relic, CloudWatch) is configured via dependency injection. Spring Boot auto-configures Micrometer with JVM, HTTP server, database, and cache metrics out of the box.

Q4. Explain the difference between Counter, Gauge, Histogram, and Summary.

Counter: monotonically increasing (total requests, total errors). Gauge: current snapshot value, can go up or down (active connections, queue depth, memory used). Histogram: bucketed distribution (request duration in time buckets); use histogram_quantile() in PromQL for percentiles. Summary: pre-computed quantiles on client side (more accurate but less flexible than histograms).

Q5. What is a trace and how does it differ from a log?

A trace represents the complete journey of a single request across multiple services and spans. It shows causality and timing (which service called which, in what order, with what latency). A log is an isolated event record from a single service at a point in time. Traces are correlated by traceId; logs include traceId for correlation. Traces answer "where is the bottleneck"; logs answer "what happened at line X".

Q6. What is a span in distributed tracing?

A span is a unit of work within a trace - a single operation with a start time, end time, and metadata. A span has: traceId (shared across all spans in a request), spanId (unique to this operation), parentSpanId (links to the calling span), operation name, service name, status (OK/ERROR), and key-value attributes (HTTP method, DB query, user ID).

Q7. What is OpenTelemetry and why is it preferred over Zipkin/Jaeger clients?

OpenTelemetry (OTel) is a CNCF standard that unifies instrumentation across traces, metrics, and logs with a single vendor-neutral SDK and protocol (OTLP). Using OTel means you instrument once and switch backends (Jaeger, Zipkin, Datadog, Honeycomb) by changing config. Direct Zipkin/Jaeger clients lock you into a specific vendor. OTel Java agent provides zero-code auto-instrumentation for Spring, JDBC, Kafka.

Q8. How does the W3C traceparent header work?

Format: traceparent: 00-{traceId}-{spanId}-{flags}. traceId is 32 hex chars (128-bit), spanId is 16 hex chars (64-bit), flags is 2 hex chars (01 = sampled). When Service A calls Service B, Service A injects its current

span as the traceparent header. Service B extracts it and creates a child span with Service A's spanId as the parentSpanId. This links all spans across services into a single trace.

Q9. What is the OpenTelemetry Java Agent and how does it work?

The OTel Java Agent (opentelemetry-javaagent.jar) is a JVM -javaagent that uses bytecode instrumentation (ASM) to automatically add tracing to frameworks (Spring MVC, WebFlux, RestTemplate, Feign, JDBC, Kafka, Redis) without code changes. It reads config from system properties or environment variables (OTEL_SERVICE_NAME, OTEL_EXPORTER_OTLP_ENDPOINT) and exports spans via OTLP to Jaeger, Zipkin, or a collector.

Q10. What is Jaeger and how does it store traces?

Jaeger is a distributed tracing platform (open-source, originally by Uber, now CNCF). It receives spans via OTLP/Thrift, stores them in a configurable backend (Cassandra, Elasticsearch, in-memory for dev), and provides a UI to search traces by service/operation, view waterfall diagrams, compare traces, and analyse latency distributions. Jaeger Collector handles ingestion; Jaeger Query serves the UI.

Q11. What is MDC (Mapped Diagnostic Context) in logging?

MDC is a ThreadLocal map in SLF4J that stores key-value pairs automatically included in every log line for that thread. Common usage: MDC.put("traceId", span.getTraceId()); MDC.put("userId", request.getUserId()). With logback-spring.xml, include MDC keys in the log pattern. The OTel-Logback bridge automatically injects the current traceId/spanId into MDC from the active span.

Q12. Why use structured (JSON) logging?

Structured JSON logs are machine-parseable: each log line is a JSON object with consistent field names (timestamp, level, message, traceId, userId, service). Log aggregation tools (Elasticsearch, Loki) can index and filter on specific fields without regex parsing. Example: {"timestamp":"2024-01-15T10:30:00Z","level":"ERROR","traceId":"a1b2c3","message":"Order not found","orderId":"123"}.

Q13. What is ELK Stack and how does it work?

ELK: Elasticsearch (search and analytics DB), Logstash (log ingestion and parsing pipeline), Kibana (visualization UI). Applications write logs to stdout; Filebeat (log shipper) tails log files and sends to Logstash; Logstash parses/transforms and indexes into Elasticsearch; Kibana provides dashboards and search. Elasticsearch indexes all fields enabling full-text search and aggregations.

Q14. How does Loki differ from Elasticsearch for log storage?

Loki indexes only log labels (service, pod, namespace) rather than the full log content. Log lines are stored as compressed chunks. This makes Loki much cheaper (10x storage reduction) but limits search to label-filtered log streams with regex/text search within lines. Elasticsearch provides powerful full-text search, aggregations, and complex analytics but at higher cost and operational complexity.

Q15. What is Promtail and how does it ship logs to Loki?

Promtail is a log shipper agent that runs as a DaemonSet on Kubernetes or as a service on each host. It discovers log files (via service discovery - same as Prometheus), attaches labels (pod name, namespace, container), and ships log lines to Loki via HTTP. It supports structured metadata extraction, pipeline stages for parsing, and handles log rotation and backpressure.

Q16. What is trace sampling and why is it needed?

In high-traffic systems, tracing every request generates enormous data. Sampling selects a subset: head-based sampling (decide at trace start, e.g., 1% of requests); tail-based sampling (decide after trace completes - keep all traces with errors, discard fast successful ones). OTel Collector supports tail-based sampling. Trade-off: reduce storage cost while retaining high-value traces (errors, slow outliers).

Q17. How do you implement log-based alerting?

In ELK: Kibana Alerting rules trigger on log query results (e.g., ERROR count > 10 in 5 min). In Grafana+Loki: Grafana Alert rules use LogQL queries. Example: sum(rate({app="order-service"} |= "ERROR" [5m])) > 5. Alerts send to PagerDuty, Slack, or email. For SLO-based alerting: burn rate alerts on error budget consumption (Prometheus recording rules).

Q18. What is the OpenTelemetry Collector and why use it?

The OTel Collector is a standalone service that receives telemetry from applications (OTLP, Jaeger, Zipkin), processes it (batch, filter, transform), and exports to multiple backends (Jaeger, Prometheus, Datadog, S3). Benefits: decouple apps from backends, add tail sampling, enrich spans with K8s metadata, reduce egress costs by filtering/sampling before export. Runs as a DaemonSet or sidecar in K8s.

Q19. How do you correlate logs with traces?

Include traceId and spanId in every log line. With OTel + Logback bridge (opentelemetry-logback-appender): the current span's traceId is automatically injected into MDC. Log the traceId in the JSON output. In Grafana: configure the Loki data source with a "derived field" on traceId that links to Jaeger, enabling a click-through from a log line to the full trace.

Q20. What are RED metrics?

RED (Rate, Errors, Duration) is a service-level metrics framework: Rate - requests per second (throughput). Errors - number or rate of failed requests. Duration - latency distribution (p50, p95, p99). RED is recommended for microservices. Compare with USE (Utilization, Saturation, Errors) which is for infrastructure/resource monitoring. Grafana dashboards typically show RED metrics per service endpoint.

Q21. What is an SLO, SLA, and SLI?

SLI (Service Level Indicator): a measurable metric (e.g., request success rate, latency p99). SLO (Service Level Objective): a target for an SLI (e.g., 99.9% success rate over 30 days). SLA (Service Level Agreement): a contractual commitment to external customers with penalties for breach. Error budget: 100% - SLO = budget for downtime/failures. When budget is exhausted, freeze feature work and prioritize reliability.

Q22. How do you implement distributed tracing without a Java agent?

Manually instrument using OpenTelemetry SDK: add opentelemetry-sdk and opentelemetry-exporter-otlp dependencies. Use @WithSpan annotation (OTel Spring extension) to auto-create spans for annotated methods. Inject Tracer bean and call tracer.spanBuilder().startSpan() for custom spans. Add OTel Spring Boot Starter for auto-instrumentation of Spring MVC and WebClient without full Java agent.

Q23. What is a Grafana dashboard and how do you build one for a microservice?

A Grafana dashboard is a collection of panels (time-series graphs, stat panels, tables, heatmaps) querying data sources (Prometheus, Loki, Jaeger). For a microservice: Panel 1: request rate (rate(http_requests_total[5m])). Panel 2: error rate %. Panel 3: latency heatmap or p99 histogram_quantile. Panel 4: JVM heap usage. Panel 5: active DB connections. Use template variables for service/env selection. Export as JSON for version control.

Q24. How does Spring Boot Actuator enable observability?

Actuator exposes management endpoints: /actuator/health (liveness/readiness, DB/Redis/external status), /actuator/prometheus (Micrometer metrics in Prometheus format), /actuator/info (app version, git commit), /actuator/env (config properties), /actuator/loggers (dynamic log level change at runtime). Configure which endpoints are exposed via management.endpoints.web.exposure.include.

Q25. What is a log correlation ID and how is it different from a trace ID?

A trace ID is part of the distributed tracing context, generated by OTel and propagated via traceparent header. A correlation ID is a simpler concept: any unique identifier (request ID, session ID) used to correlate related log lines. Often the trace ID is used as the correlation ID. Some systems have multiple correlation IDs: trace ID (for tracing), request ID (per HTTP request), session ID (per user session).

Q26. How do you monitor JVM performance with Micrometer?

Micrometer auto-registers JVM metrics with Spring Boot Actuator: jvm.memory.used/max (heap and non-heap), jvm.gc.pause (GC duration by cause), jvm.threads.live/daemon/peak, process.cpu.usage, jvm.classes.loaded. Grafana JVM dashboard (ID 4701) visualises these. Alert on: heap usage > 85%, frequent Full GC, thread count growth (thread leak), CPU > 80%.

Q27. What is the difference between push-based and pull-based metrics?

Pull-based (Prometheus): Prometheus scrapes targets at configured intervals. Simple, targets are passive; Prometheus controls the scrape rate; easy to detect if a target is down (scrape fails). Push-based (StatsD, Graphite, Datadog Agent): services push metrics to a central collector. Better for short-lived jobs (completed before next scrape), firewall-friendly (no inbound connections needed). Prometheus supports push via Pushgateway for batch jobs.

Q28. How do you implement dynamic log levels at runtime?

Spring Boot Actuator /actuator/loggers endpoint supports GET (current levels) and POST (change level) without restart. POST /actuator/loggers/com.example.order with body {"configuredLevel": "DEBUG"} enables debug logging for that package. Useful for production diagnosis without redeployment. Revert by setting configuredLevel to null (inherits parent level). Micrometer can also configure log-level-based metrics.

Q29. What is a flame graph and how does it help performance analysis?

A flame graph visualises the call stack of a profiled application: width = CPU time spent in that function/method, height = call depth. In distributed tracing: Jaeger renders a span tree as a timeline. Java profilers (async-profiler, JFR) generate CPU flame graphs showing which methods consume most CPU. Combined with traces, flame graphs help identify hot paths at both service (trace) and code (profiler) levels.

Q30. What is cardinality in metrics and why does it matter?

Cardinality is the number of unique time series created by label combinations. High cardinality (e.g., labeling by user_id, request_id, or full URL path) creates millions of time series, overwhelming Prometheus storage and query performance. Rule: labels should have bounded, low-cardinality values (service, method, status_code, endpoint - without dynamic IDs). Use tracing (not metrics) for per-request detail.

Q31. How do you implement alerting in Prometheus?

Define alert rules in Prometheus rules files: ALERT HighErrorRate IF rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m]) > 0.05 FOR 5m LABELS {severity="critical"} ANNOTATIONS {summary="Error rate above 5%"}. Prometheus evaluates rules every evaluation_interval (default 1m). Firing alerts are sent to Alertmanager, which routes to

PagerDuty/Slack/email with deduplication and silence support.

Q32. What is exemplar in Prometheus and how does it bridge metrics to traces?

Exemplars are trace references attached to metric observations. For a histogram bucket, an exemplar stores a representative traceId for a request that fell in that bucket. In Grafana: click on a data point in a latency histogram to jump directly to a Jaeger trace for that specific request. Requires OpenMetrics format (not standard Prometheus text), Prometheus 2.26+, and Micrometer 1.7+ with OTLP bridge.

Q33. How do you configure OpenTelemetry sampling?

Configure via -Dotel.traces.sampler=: always_on (100% sampled, dev), always_off (0%, disabled), traceidratio (e.g., 0.01 = 1%), parentbased_traceidratio (respects parent sampling decision). For tail-based sampling, use OTel Collector with tail_sampling processor: define policies (always_sample on ERROR status, probabilistic 1% otherwise). Tail sampling requires the Collector to buffer and evaluate complete traces.

Q34. What is the difference between Jaeger and Zipkin?

Both are distributed tracing backends. Jaeger: supports OTLP natively, better scalability (Cassandra/Elasticsearch backend), more advanced UI (compare traces, critical path), CNCF project, actively developed. Zipkin: older, simpler, broader language support historically, uses B3 propagation headers (supported by OTel). For new projects, prefer Jaeger or a managed service (Datadog, Honeycomb). OTel supports both via exporters.

Q35. How does tracing work with asynchronous Kafka consumers?

OTel Java agent auto-instruments Kafka consumers: it extracts the traceId from Kafka message headers (injected by the producer), creates a child span for the consumer processing, and links it to the original producer trace. For manual instrumentation: extract context from ConsumerRecord headers using W3CTraceContextPropagator, create a span with Context.makeCurrent(). This creates a linked trace across service boundaries via Kafka.

Q36. What is slow query logging and how do you integrate it with observability?

Configure the database to log queries exceeding a threshold: PostgreSQL log_min_duration_statement=1000; MySQL slow_query_log=1, long_query_time=1. Micrometer auto-instruments JDBC with spring.datasource.hikari metrics (connection pool stats). For OTel DB tracing: spans for SQL queries include the statement, duration, and db.system attribute. Correlate slow queries with trace IDs to identify which request triggered the slow query.

Q37. What is the OTLP protocol?

OpenTelemetry Protocol (OTLP) is the standard wire format for telemetry data in the OTel ecosystem. Supports gRPC (default, port 4317) and HTTP/Protobuf (port 4318). All OTel SDKs export via OTLP to the OTel Collector or directly to OTLP-compatible backends (Jaeger, Grafana Tempo, Datadog, Honeycomb). Using OTLP ensures portability - change backend by reconfiguring the Collector, not the application.

Q38. How do you implement SLO-based alerting with Prometheus?

Define recording rules for error budget: record: job:request_error_rate:ratio_rate5m -> expr: sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m])). Alert on burn rate: error_budget_burn_rate_1h > 14.4 means you're consuming the monthly budget 14.4x faster than sustainable. Multi-window, multi-burn-rate alerting (1h and 6h windows) reduces false positives and alert fatigue.

Q39. What is Grafana Tempo?

Grafana Tempo is a cost-efficient distributed tracing backend that stores traces in object storage (S3, GCS). It receives spans via OTLP, Jaeger, or Zipkin. Key feature: TraceQL query language for filtering traces. Integrates natively with Grafana for unified observability: click from a Loki log line to a Tempo trace to a Prometheus metric. Cheaper than Jaeger with Elasticsearch backend for high-volume tracing.

Q40. How do you handle log storage costs in production?

Strategies: (1) Structured logs with selective fields - only log what you need. (2) Log sampling - only log a percentage of successful requests (always log errors). (3) Log levels per environment - DEBUG in staging, INFO in prod, ERROR-only in high-volume services. (4) Retention policies - 7 days hot (Elasticsearch), 30 days warm, 90 days cold (S3). (5) Loki instead of Elasticsearch (10x cheaper). (6) Aggregated metrics instead of per-request logs for throughput data.

Q41. What is a health check endpoint and what should it verify?

/actuator/health returns: UP/DOWN/OUT_OF_SERVICE with component status. Verify: database connectivity (DataSourceHealthIndicator), Redis ping, Elasticsearch cluster health, disk space. Kubernetes liveness probe (is the process running): /actuator/health/liveness. Readiness probe (can the pod serve traffic): /actuator/health/readiness (includes external dependency checks). Startup probe: longer timeout for slow-starting JVMs.

Q42. What is distributed log aggregation and why is a central store needed?

In a microservices cluster, each service instance writes logs to its own stdout/file. Without aggregation, you would need to SSH to each pod to read logs. Centralized log aggregation (ELK, Loki) collects logs from all instances, enriches with metadata (pod name, namespace, version), and stores them searchable in one place. Enables: cross-service log correlation by traceId, production debugging without cluster access, compliance log retention.

Q43. How does the OTel-Logback bridge work?

The opentelemetry-logback-appender-1.0 library bridges OTel context to SLF4J/Logback: it registers an SLF4J MDC adapter that automatically populates MDC with the current span's traceId, spanId, and traceFlags when a log statement is made inside an active OTel span. Result: every log line within a traced request automatically includes the traceId without manual MDC.put() calls.

Q44. What is a black box vs white box monitoring?

Black box monitoring tests system behaviour from the outside (like a user): HTTP health checks, synthetic transactions, uptime checks. Tells you "the system is down". White box monitoring uses internal metrics, logs, traces from inside the system: JVM metrics, DB query count, queue depth. Tells you "why it is slow". Both are needed: black box for user-visible SLO monitoring, white box for root-cause analysis.

Q45. How do you use Micrometer for Kafka consumer lag monitoring?

Micrometer Kafka binder (spring-kafka) exposes: kafka.consumer.records-lag (current lag per partition), kafka.consumer.fetch-latency-avg, kafka.consumer.records-consumed-rate. Consumer lag measures the difference between the last produced offset and the last consumed offset per partition - a growing lag indicates the consumer cannot keep up with producers. Alert on lag > threshold for critical topics.

Q46. What are log levels and when should each be used?

TRACE: very fine-grained info, per-method entry/exit, rarely enabled. DEBUG: diagnostic information for developers, disabled in production. INFO: normal operational events (service started, user logged in,

scheduled job ran). WARN: unexpected but handled situations (deprecated API called, retry succeeded, configuration default used). ERROR: failures requiring action (DB unreachable, uncaught exception, data integrity violation). FATAL: application cannot continue (very rare, usually triggers shutdown).

Q47. What is the difference between availability monitoring and performance monitoring?

Availability monitoring: is the service up and responding? Binary status. Metrics: uptime %, error rate, health check pass/fail. Performance monitoring: how fast and efficiently is the service responding? Continuous measurements. Metrics: latency percentiles, throughput, saturation (queue depth, connection pool usage). Both are needed: availability tells you if users can use the system; performance tells you if users have a good experience.

Q48. How do you trace a slow database query to a specific API request?

With OTel auto-instrumentation: each SQL query executed within a traced HTTP request creates a child span with db.statement, db.system, db.operation attributes. In Jaeger, find the trace for the slow API request and expand child spans to see which SQL statements were executed, their duration, and which were slow. Combine with slow query log correlation: the SQL in the span matches the slow query log entry, both containing the traceId.

Q49. What is canary analysis using observability data?

During a canary deployment, route 5% of traffic to the new version. Compare RED metrics (error rate, latency) between canary and baseline using Prometheus queries filtered by version label. If canary error rate > baseline * 1.1 (10% regression), automatically roll back. Tools: Argo Rollouts + Kayenta perform automated canary analysis against Prometheus/Datadog metrics, preventing regressions from reaching 100% traffic.

Q50. How do you implement distributed tracing for gRPC services?

OTel Java Agent auto-instruments gRPC: it intercepts gRPC client and server calls, creates spans, and propagates trace context via gRPC metadata headers using the grpc-trace-bin (binary) or traceparent (text) metadata key. For manual instrumentation: use gRPC interceptors (ClientInterceptor, ServerInterceptor) to extract/inject trace context. Service meshes (Istio) can also handle trace propagation at the proxy layer.

Q51. What is the role of an observability platform like Datadog or New Relic?

Managed observability platforms provide an integrated solution: metrics, traces, logs, and dashboards in one product with no infrastructure to manage. They offer: auto-instrumentation agents, AI-powered anomaly detection, service maps, APM (Application Performance Monitoring), RUM (Real User Monitoring), and SLO tracking. Trade-off: vendor lock-in and per-host/per-GB pricing can be expensive at scale. OpenTelemetry mitigates vendor lock-in.

Q52. What is continuous profiling and how does it complement tracing?

Continuous profiling collects CPU/memory/thread profiles from production applications continuously (unlike point-in-time profiling). Tools: Pyroscope, Grafana Pyroscope, async-profiler, JFR. Profiles show which code paths consume most resources system-wide. Tracing shows which requests are slow (top-down). Profiling shows why specific code is slow (bottom-up). Grafana integrates Pyroscope with Tempo: click from a slow trace span to the flame graph for that time period.